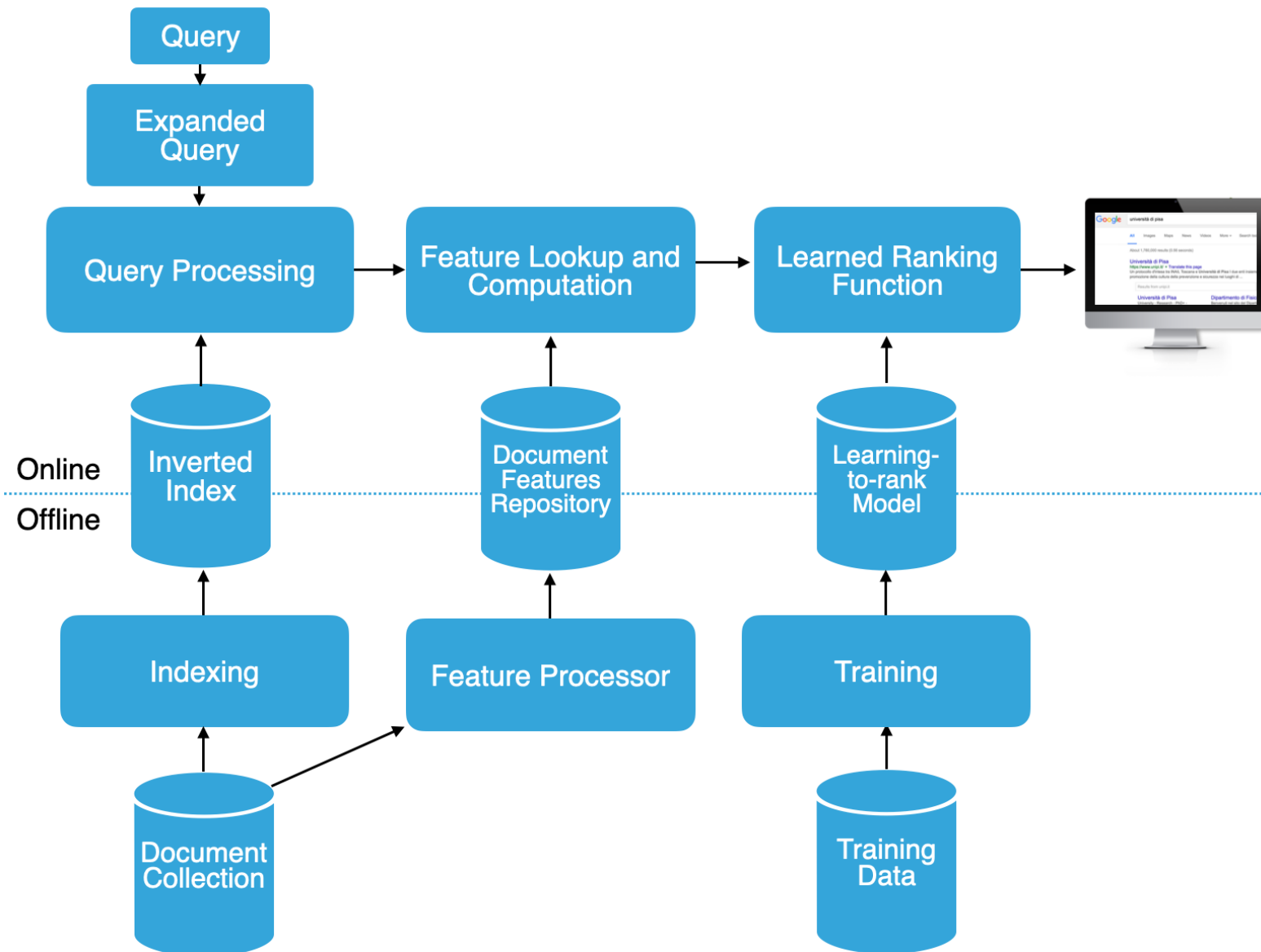


EVALUATION

How can we evaluate the quality of a SE?



How can we evaluate the quality of a SE?

- Questions about *efficiency*
 - How fast does it index a collection?
 - Number of documents/hour
 - Incremental indexing – the SE adds 10K products/day
 - How fast does it search?
 - Latency and CPU needs for processing queries on 5 million documents
- Questions about *effectiveness/quality* of side services
 - Does it recommend good related products to buy?
- It says nothing about the intrinsic *effectiveness/quality* of the SE
 - We want users to be *happy* with the search experience, in particular, with the SERP prepared for each search query

How do you tell if users are happy?

- Search results get clicked a lot
 - Misleading titles/summaries can cause users to click
 - When “no clicks” is a good news?
- Users' buys after using the search engine
 - Users spend a lot of \$ after using the search engine
- Repeated visitors/buyers
 - Do they come back within a week/month/... ?
- Dwell time
 - How much time users spend after clicking a link on a SERP page before clicking back to the SERP results

... is this still valid for ChatGPT?

Happiness: elusive to measure

- Most common proxy: *relevance* of search results
 - But how do you measure relevance?
- Pioneered by Cyril Cleverdon in the Cranfield Experiments

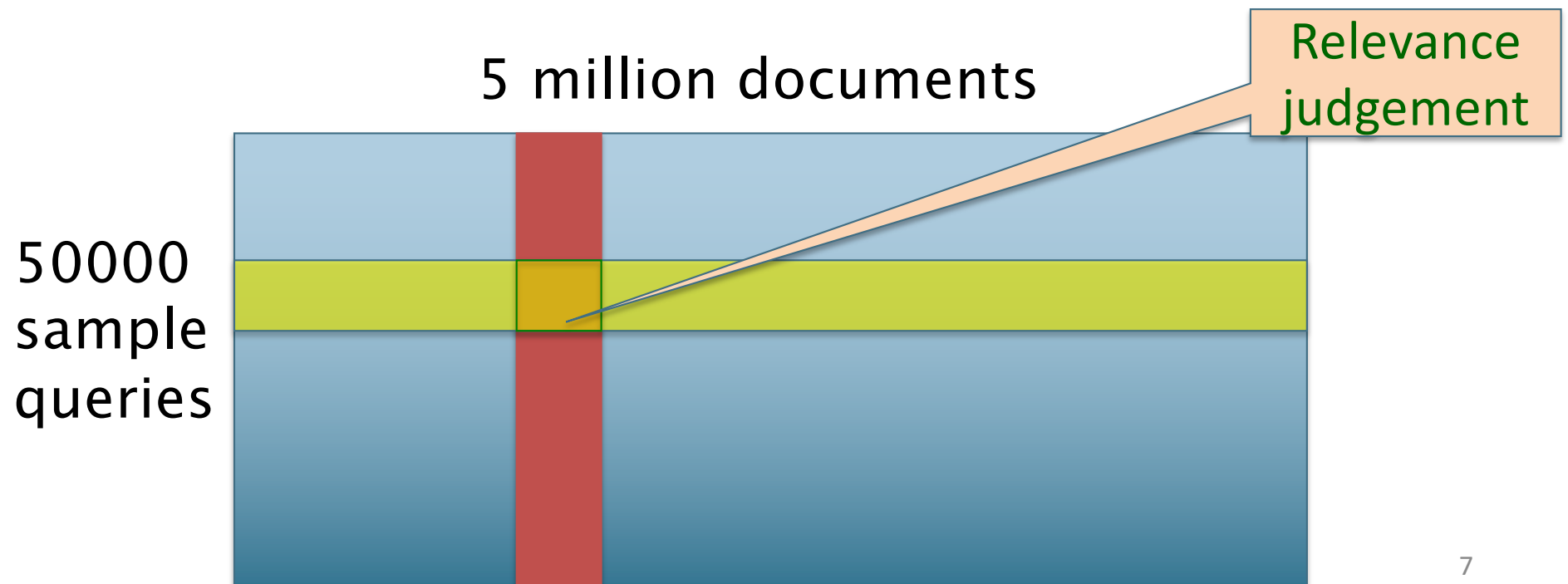


Measuring relevance

- Three elements:
 1. A benchmark document collection
 2. A benchmark suite of queries
 3. An assessment of either Relevant or Nonrelevant for each query and each document

We want to measure the quality of a new search algorithm

- Benchmark documents
- Benchmark query suite
- Judgments of document relevance for each query



Relevance judgments

- Binary (relevant vs. non-relevant) in the simplest case, more nuanced (0, 1, 2, 3 ...) in others
- What are some issues already?
- $5 \text{ million} \times 50\text{K} = 2.5 \times 10^{11} = 0.25 \times 10^{12}$
 - 1/4 trillion judgments
 - if each judgment took a human 2.5 seconds, we need about 6.25×10^{11} seconds (~170 M hours / person)
- Indeed, we limit the number of judgements to a subset of the 5 M docs

Crowd source relevance judgments?

- Present query-document pairs to low-cost workers on online **crowd-sourcing platforms** (e.g., Amazon Mechanical Turk)
 - cheaper than hiring qualified assessors
- Lots of literature on using crowd-sourcing for such tasks
- Main takeaway – you get some signal, but the variance in the resulting judgments is very high

... Any other idea? 😊

What else?

- Still need test queries
 - These queries must be relevant/suitable to docs available
 - Must be representative of actual user needs
 - Random query terms from the documents is not generally a good idea
 - **Sample from query logs** if available
- Classically (non-Web)
 - Low query rates – not enough query logs
 - Experts hand-craft “user needs” (“topics” in TREC terminology) and associated “queries”

Some public test Collections

TABLE 4.3 Common Test Corpora

<i>Collection</i>	<i>NDocs</i>	<i>NQrys</i>	<i>Size (MB)</i>	<i>Term/Doc</i>	<i>Q-D RelAss</i>
ADI	82	35			
AIT	2109	14	2	400	>10,000
CACM	3204	64	2	24.5	
CISI	1460	112	2	46.5	
Cranfield	1400	225	2	53.1	
LISA	5872	35	3		
Medline	1033	30	1		
NPL	11,429	93	3		
OSHMED	34,8566	106	400	250	16,140
Reuters	21,578	672	28	131	
TREC	740,000	200	2000	89-3543	» 100,000

Typical
TREC

TREC

- Text REtrieval Conference (TREC):
 - Aims: creating an evaluation infrastructure required for large-scale testing of information retrieval (IR) technology.
 - This includes research on best methods for evaluation as well as development of the evaluation materials themselves.
 - “Retrieval technology” is broadly interpreted to include a variety of techniques that enable and/or facilitate access to information not specifically structured for machine use.
 - Sponsored by Nat. Inst. of Standards and Technology (NIST)
 - TREC is organized around a set of focus areas called “**tracks**”. A track has a motivating use case, which is generally an abstraction of a user task
(<https://trec.nist.gov/pubs/trec32/xref.html>)

TREC

- Test collections consist of three parts:
 1. a set of documents
 2. a set of information needs (called topics in TREC)
 3. relevance judgments, an indication of which documents should be retrieved in response to which topics.
- We call the result of a retrieval system executing a task on a test collection a **run**.

Document sampling and relevance judgments

- “Pooling” is the technique used in early TRECs for selecting the sample of documents for the human assessor to judge.
 - In pooling, the **top results** from a set of **runs** are combined to form the pool and only those documents in the pool are judged.
 - **Runs are subsequently evaluated assuming that all unpooled (and hence unjudged) documents are not relevant** 😞
- Several TREC tracks have investigated new ways of sampling from very large documents sets to obtain judgment sets that support fair evaluations.

Now we have the basics of a benchmark

- Let's review some evaluation measures
 - *Precision*
 - *Recall*
 - DCG
 - ...

Evaluating an IR system

- Note: **user need** must be translated into a **query**
- Relevance is assessed w.r.t. the **user need**, *not* the **query**
 - Information need: *My swimming pool bottom is becoming black and needs to be cleaned.*
 - Query: ***pool cleaner***
- TREC participants are (usually) free to use any method to create **queries** from **topic** statements, either automatic or manual ones

Unranked Measures: Precision and Recall

■ Binary assessments

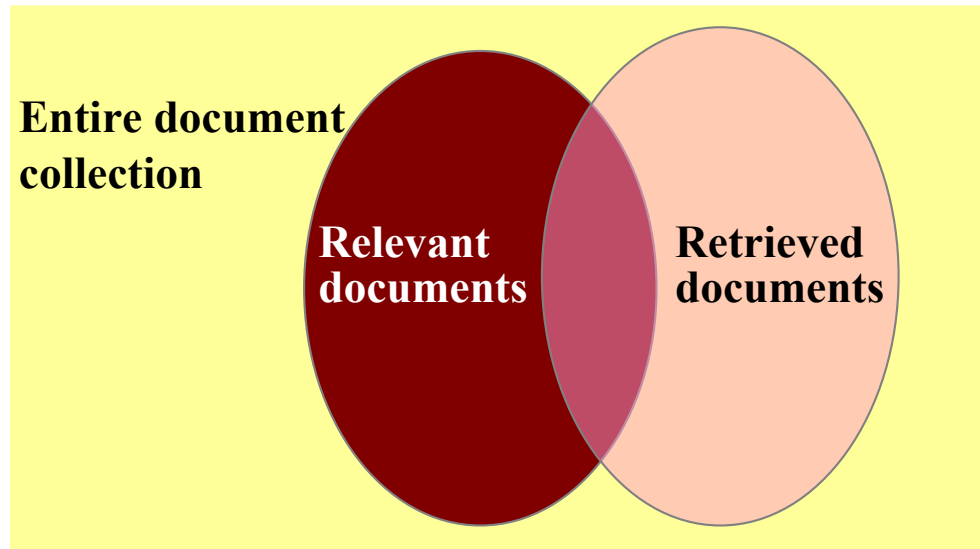
Precision: fraction of retrieved docs that are relevant
 $P(\text{relevant retrieved} \mid \text{retrieved})$

Recall: fraction of relevant docs that are retrieved
 $P(\text{relevant retrieved} \mid \text{relevant})$

	Relevant	Nonrelevant
Retrieved	tp	fp
Not Retrieved	fn	tn

- Precision $P = tp / (tp + fp)$
- Recall $R = tp / (tp + fn)$

Precision and Recall



irrelevant	retrieved & irrelevant	Not retrieved & irrelevant
relevant	retrieved & relevant	not retrieved but relevant
	retrieved	not retrieved

$$recall = \frac{\text{Number of relevant documents retrieved}}{\text{Total number of relevant documents}}$$

$$precision = \frac{\text{Number of relevant documents retrieved}}{\text{Total number of documents retrieved}}$$

F-Measure or F-Score

- F-Measure is the *harmonic mean* of Precision and Recall:

$$F = \frac{1}{0.5 \frac{1}{P} + 0.5 \frac{1}{R}} = 2 \frac{PR}{P + R}$$

- The harmonic mean is always less than or equal to the arithmetic mean and the geometric mean.
 - When the values of two numbers differ greatly, the **harmonic mean is closer to their minimum** than to their arithmetic mean
 - Consider an example with *recall* = 100% and *precision* very low: *F* becomes close to *precision*

F-Measure or F-Score

- If we give weight α to P and $(1-\alpha)$ to R, with $\alpha = 1/(1+\beta^2)$, we have **weighted F-Measure**:
- Values of $\beta < 1$ emphasize **precision** (approaches precision), while values of $\beta > 1$ emphasize **recall** (approaches recall)

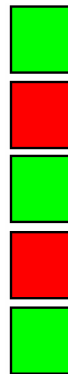
$$F_{\beta} = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} = (\beta^2 + 1) \frac{PR}{\beta^2 P + R}$$

Rank-Based Measures

- Binary relevance
 - Mean Average Precision (MAP)
 - Precision@K ($P@K$)
 - Mean Reciprocal Rank (MRR)
- Multiple levels of relevance
 - Normalized Discounted Cumulative Gain (NDCG)

MAP (Mean Average Precision)

- Consider the rank position of each **relevant** doc, where **recall** increases, $K_1, K_2, \dots K_R$
- Compute Precision@K for each $K_1, K_2, \dots K_R$
- **AP** (avg precision) for a query with only $R=3$ relevant docs returned

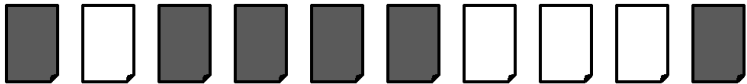


has an AP of $\frac{1}{3} \cdot \left(\frac{1}{1} + \frac{2}{3} + \frac{3}{5} \right) \approx 0.76$

- **MAP** is the mean AP across **multiple** queries/rankings
 - Still provides a single-figure measure of quality for a set of queries

Average Precision

 = the relevant documents

Ranking #1	
Recall	0.17 0.17 0.33 0.5 0.67 0.83 0.83 0.83 0.83 1.0
Precision	1.0 0.5 0.67 0.75 0.8 0.83 0.71 0.63 0.56 0.6
Ranking #2	
Recall	0.0 0.17 0.17 0.17 0.33 0.5 0.67 0.67 0.83 1.0
Precision	0.0 0.5 0.33 0.25 0.4 0.5 0.57 0.5 0.56 0.6

$$\text{Ranking \#1: } (1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$$

$$\text{Ranking \#2: } (0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$$

MAP

 = relevant documents for query 1

Ranking #1

										
Recall	0.2	0.2	0.4	0.4	0.4	0.6	0.6	0.6	0.8	1.0
Precision	1.0	0.5	0.67	0.5	0.4	0.5	0.43	0.38	0.44	0.5

 = relevant documents for query 2

Ranking #2

										
Recall	0.0	0.33	0.33	0.33	0.67	0.67	1.0	1.0	1.0	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.33	0.43	0.38	0.33	0.3

$$\text{average precision query 1} = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$

$$\text{average precision query 2} = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$\text{mean average precision} = (0.62 + 0.44)/2 = 0.53$$

Mean average precision (MAP)

- MAP is **macro-averaging**: each query counts equally
 - MAP is the arithmetic mean of the AP values
 - it weighs each query (information need) equally, even if many documents are relevant to some queries, whereas very few to others
- One of the most used measures in research papers for **binary relevance**

Precision@K

- The above measures are good for web search?
 - They factor in precision at **all** recall levels
 - In Web Search the total number of relevant docs could not be known, or could be many
- What matters in Web Search is rather how many good results there are on the first page pages (e.g., one/three pages, 10/30 results)
 - **P@k** (Precision at k)

Precision@K

- Set a rank threshold K
- Compute % relevant in top K
- Ignores documents ranked lower than K
- $P@K = \frac{\sum \text{relevant docs in top-K}}{K}$

- Ex:
 - $\text{Prec}@3$ of $2/3 = 0.66$
 - $\text{Prec}@4$ of $2/4 = 0.5$
 - $\text{Prec}@5$ of $3/5 = 0.6$



Note that $\text{Prec}@4$ is smaller than $\text{Prec}@3$

- In a similar fashion, $\text{Recall}@K$

MAP@K

- P@K does not average well over a set of queries
 - The **total number of relevant documents** for a query has a strong influence on AP@k per query
- Anyway, AP@K and MAP@K are largely used for Web Search and Recommender Systems

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot rel(q_i, k) \quad K_i = \sum_{k=1}^K rel(q_i, k)$$

$$rel(q_i, k) = \begin{cases} 1 & \text{if item at the } k_{th} \text{ rank is relevant} \\ 0 & \text{otherwise} \end{cases}$$

$$MAP@K(Q) = \frac{1}{|Q|} \sum_{q_i \in Q} AP@K(q_i)$$

What if relevant doc(s) are not on the returned result list?

- Suppose there's only one Relevant Document
- Scenarios:
 - known-item search
 - navigational queries
 - looking for a fact
- Search duration for a user $\approx$ Rank of the correct answer
 - The *rank* measures the user's effort to find the result
 - Mean Reciprocal Rank (MRR) is a measure of this effort, and generalizes this concept

Mean Reciprocal Rank

- Consider **rank position**, $rank_i$, **of the first relevant doc** returned for query $q_i \in Q$
 - Could be – only clicked doc

- Reciprocal Rank (RR) score = $\frac{1}{rank_i}$

- MRR is the mean RR across multiple queries

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}.$$

BEYOND BINARY RELEVANCE



information retrieval unipi



All

Images

Videos

News

Web

Books

Finance

Tools



DidaWiki

<http://didawiki.cli.di.unipi.it> › [magistraleinformatica](#) › [start](#) :

Information Retrieval

General **Information** · Teacher : Paolo Ferragina · Course ID: 289AA · Semester: first · CFU: 6
(theory + lab) · Language: English · News about this course will be ...



uniPi

<https://esami.unipi.it> › [programma](#) · [Translate this page](#) :

information retrieval - Valutami - Università di Pisa

Gli studenti alla fine del percorso didattico saranno in grado di progettare e valutare motori di ricerca e IR tool, adottando soluzioni algoritmiche allo stato ...



uniPi

<https://esami.unipi.it> › [esami2](#) › [pdfProgCorsoStu](#) [PDF](#) :

information retrieval

Studio, progetto e analisi di sistemi di Information Retrieval che risultano efficienti ed efficaci nel processare, analizzare, memorizzare, ricercare,.

Discounted Cumulative Gain

- Popular measure for evaluating web search and related tasks
- Two assumptions:
 - Highly relevant documents are more useful than marginally relevant documents
 - The lower the ranked position of a relevant document, the less useful it is for the user, since it is less likely to be examined

Discounted Cumulative Gain

- Uses *graded relevance* as a measure of usefulness, or *gain*, from examining a document
- *Gain is accumulated* starting at the top of the ranking and may be reduced, or *discounted*, at *lower ranks*
- Typical discount is $1/\log(\text{rank})$
 - With base 2, the discount at rank 4 is $1/2$, and at rank 8 it is $1/3$

Summarize a Ranking: DCG

- **Relevance judgments** on a scale of $[0, m]$, $m > 2$
- Cumulative Gain (**CG**) at rank n
 - Let the **judgements** of the n documents be $r_1, r_2, \dots, r_n$ (in ranked order)
 - $CG = r_1 + r_2 + \dots + r_n$
- Discounted Cumulative Gain (**DCG**) at *rank* n
 - The gain is discounted if the relevant docs appear at a ranks > 1
 - $DCG = r_1 + r_2 / \log_2 2 + r_3 / \log_2 3 + \dots + r_n / \log_2 n$
 - We may use any base for the logarithm

Discounted Cumulative Gain

- DCG is the total gain accumulated at a particular rank p :

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2 i}$$

- Alternative formulation (with $\log_2$), where the high relevance judgments become much more important:

$$DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log(1+i)}$$

- used by some web search companies
- emphasis on retrieving highly relevant documents

DCG Example

- 10 ranked documents judged on 0-3 relevance scale:

3, 2, 3, 0, 0, 1, 2, 2, 3, 0

- $\frac{rel_i}{\log_2 i}$: 3, 2/1, 3/1.59, 0, 0, 1/2.59, 2/2.81, 2/3, 3/3.17, 0

■ = 3, 2, 1.89, 0, 0, 0.39, 0.71, 0.67, 0.95, 0

- $DCG@p, \quad p=1,2,3,\dots,10$

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2 i}$$

3, 5, 6.89, 6.89, 6.89, 7.28, 7.99, 8.66, 9.61, 9.61

Summarize a Ranking: NDCG

- Normalized Discounted Cumulative Gain (**NDCG**) at rank n
 - **Normalize DCG** at rank n by the DCG value at rank n of the **ideal ranking**
 - The ideal ranking would first return the documents with the highest relevance level, then the next highest relevance level, etc.
 - There are ties ...
- Normalization useful for contrasting queries with varying numbers of relevant results
- NDCG is now quite popular in evaluating Web search

NDCG Example

- Perfect ranking:

- 3, 3, 3, 2, 2, 2, 1, 0, 0, 0

$$DCG_p = rel_1 + \sum_{i=2}^p \frac{rel_i}{\log_2 i}$$

- **Ideal DCG** values:

- 3, 6, 7.89, 8.89, 9.75, 10.52, 10.88, 10.88, 10.88, 10.88

- The **actual rank** of 10 documents as returned by the search engine:

- 3, 2, 3, 0, 0, 1, 2, 2, 3, 0

- **Actual DCG**:

- 3, 5, 6.89, 6.89, 6.89, 7.28, 7.99, 8.66, 9.61, 9.61

- NDCG values (divide **actual** DCG by **ideal** DCG) :

- 1, 0.83, 0.87, 0.76, 0.71, 0.69, 0.73, 0.8, 0.88, 0.88
- $NDCG \leq 1$ at any rank position

NDCG - Example

4 documents: d_1, d_2, d_3, d_4

i	Ground Truth		Ranking Function ₁		Ranking Function ₂	
	Document Order	r_i	Document Order	r_i	Document Order	r_i
1	d4	2	d3	2	d3	2
2	d3	2	d4	2	d2	1
3	d2	1	d2	1	d4	2
4	d1	0	d1	0	d1	0
	NDCG _{GT} =1.00		NDCG _{RF1} =1.00		NDCG _{RF2} =0.9203	

$$DCG_{GT} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

$$DCG_{RF1} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

$$DCG_{RF2} = 2 + \left(\frac{1}{\log_2 2} + \frac{2}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.2619$$

$$MaxDCG = DCG_{GT} = 4.6309$$

Recap

- Benchmarks consist of
 - Document collection
 - Query set
 - Assessment methodology
- Assessment methodology can use raters, user clicks, or a combination
 - These get quantized into a *goodness measure* – Precision/NDCG etc.
 - Different engines/algorithms compared on a benchmark together with a goodness measure